

StorePilot

Store Management System & Online Shopping App

Student Name: _____

Student ID: _____

School Name: _____

Supervisor: _____

Alternative: **Smart Phones Programming (5 units)**

Submission Date: _____

Table of Contents

1. Introduction	3
2. Project Architecture / Structure	8
3. Project Implementation	13
4. User Guide	21
5. Personal Reflection / Summary	25
6. Bibliography	28
7. Appendices	29

1. Introduction

1.1 Project Background

Project Name

StorePilot

Short Description

StorePilot is a dual-role Android application that combines two worlds in one: a complete store management system for the owner and staff, and an online shopping app for customers. The system is built around the MVVM (Model-View-ViewModel) architecture, uses Firebase Authentication for user verification, and Firebase Firestore as the cloud database.

Target Audience

- Small and medium business owners who want digital management of inventory, sales, tasks, and seasonal collections.
- Store managers and employees who need a fast, convenient daily work tool.
- Customers who want to purchase products from the store through a secure, easy-to-use app.

Reasons for Choosing This Topic

- Combines diverse fields: business management, security (password encryption, owner identity verification via secret code), modern UX, and cloud computing.
- High relevance: many small stores still manage inventory and sales manually.
- Opportunity to implement a wide range of advanced topics: Firebase, AlarmManager, Notifications, Fragments, Threads, MVVM, and more.

1.2 Research Process

Domain Knowledge Research

A review of existing Store Management / POS systems was conducted, including Square, Shopify, and Lightspeed. It was found that most of these systems are expensive and complex for small businesses, and none offer a unified staff + customer interface in a single app.

Review of Existing Applications

E-commerce apps such as Amazon, Shopify Customer App, and Wix Owner were examined. The conclusion: combining a management interface with a customer shopping interface in one app is uncommon in the market.

Technologies Not in the Curriculum

- **Firestore** - Google's NoSQL cloud database. Supports real-time data syncing and automatic UI updates via snapshot listeners.
- **Authentication** - Google's user authentication service. Supports email/password sign-in.
- **AlarmManager** - Android component for executing code at fixed time intervals, even when the app is in the background.
- **PBKDF2WithHmacSHA256** - Advanced password hashing algorithm with a random salt, providing strong protection against brute-force attacks.

1.3 Main Challenges

- **Complex permission management:** four staff roles (Owner, Store Manager, Shift Manager, Employee) plus Customer, each with different permissions.
- **Owner identity verification:** implementing a secret code mechanism (777) that appears as a dialog when selecting the Owner role during registration.
- **Real-time notifications:** when stock falls below the threshold, the system sends a notification even when the app is in the background, using AlarmManager and BroadcastReceiver.
- **LiveData chaining:** views combining multiple data sources (favorites + products, cart + products) required careful observer management.
- **Firestore queries without composite indexes:** avoided composite index errors by sorting data in Java instead of using Firestore orderBy() on subcollections.

Need the Project Addresses

Small stores need a simple, accessible, and affordable management tool that also allows customers to shop directly. StorePilot provides a unified solution for both sides in a single application.

1.4 Innovations and Adaptations

- **Owner authentication code:** an innovative method using a secret code (777) to verify the store owner before registration.
- **Periodic AlarmManager:** checks stock every hour and automatically sends a notification when a product drops below 5 units.
- **Test Notification mode:** a dedicated button in the More menu that instantly triggers the notification system without waiting for the alarm.
- **Automatic sale creation:** when an order status is changed to DELIVERED, the system automatically creates a sale record and updates revenue figures.

1.5 Knowledge Domain Description

Required Objects

Object	Description
User	System user (Owner / Store Manager / Shift Manager / Employee / Customer)

Product	Item in the store inventory
Sale	A recorded sale transaction
Order	A customer order
OrderItem	A single item within an order
CartItem	An item in a customer's shopping cart
Favorite	A product saved in a customer's wishlist
Task	A task assigned to an employee
Season	A seasonal collection or promotion period
SupportMessage	A chat message between customer and staff

Data Types

- String: names, emails, addresses, roles, statuses
- int: IDs, quantities
- double: prices, totals
- long: timestamps in UNIX format
- boolean: statuses (isActive, isFavorite)

Data Representation

- ArrayList: dynamic lists of objects displayed in RecyclerView components.
- LiveData<List<T>>: reactive observation of Firestore data for automatic UI updates.
- Firestore Collections: cloud storage as nested documents (NoSQL).
- Subcollections: carts/{userId}/items, favorites/{userId}/items, orders/{orderId}/items, support/{customerId}/messages.

Operations on Data

- Insert: each ViewModel calls a Repository that executes add() to Firestore.
- Read: snapshot listeners from Firestore wrap LiveData - the UI updates automatically on any change.
- Update: flow is View -> ViewModel -> Repository -> Firestore document.update().
- Delete: user confirmation required; the document is removed from Firestore.

2. Project Architecture / Structure

2.1 Welcome Screen Description - WelcomeActivity

The welcome screen is the app's entry point. On launch it checks whether the user is already logged in (`SessionManager.isLoggedIn`). If yes, it routes them directly based on their role (Customer -> `CustomerMainActivity`; Owner/Manager/Employee -> `MainActivity`). If not, it displays two buttons: Sign In and Sign Up.

2.2 Screen Descriptions

1. WelcomeActivity

Role: Entry point

Description: App logo and two buttons. `btnLogin` -> `LoginActivity`; `btnRegister` -> `RoleSelectionActivity`.

2. RoleSelectionActivity

Role: Role selection during registration

Description: Two options: Owner or Customer. `btnRoleOwner` opens a dialog requesting secret code 777. `btnRoleCustomer` navigates directly to `RegisterActivity` with the Customer role.

3. LoginActivity

Role: Login screen

Description: Fields: `etEmail` (email address), `etPassword` (password). `btnLogin` verifies credentials via Firebase Auth, updates `SessionManager`, and routes by role.

4. RegisterActivity

Role: Registration screen

Description: Fields: `etFullName`, `etUsername`, `etEmail`, `etPhone`, `etPassword`, `etConfirmPassword`. Encrypts password with `PBKDF2WithHmacSHA256`, registers with Firebase Auth, saves profile to Firestore.

5. MainActivity

Role: Main screen for staff (Owner / Manager / Employee)

Description: `BottomNavigationView` with tabs: Dashboard, Inventory, Sales, Tasks, More. More menu: Orders, Support Inbox, Seasons, Admin, Test Notification, Logout. Starts `AlarmManager` for hourly stock checks.

6. CustomerMainActivity

Role: Main screen for customers

Description: BottomNavigationView with tabs: Shop, Cart, Orders, Favorites, Support. Sign Out lock icon in the top bar.

7. AddEditProductActivity

Role: Add / Edit product (Owner only)

Description: Fields: product name, category, size, color, quantity, selling price, cost price. btnSaveProduct saves to Firestore. Access restricted by PermissionManager.

8. AddSaleActivity

Role: Record a manual sale

Description: Fields: product ID, quantity, total price, notes. Accessible to all staff roles.

9. AddEditSeasonActivity

Role: Season / Collection management

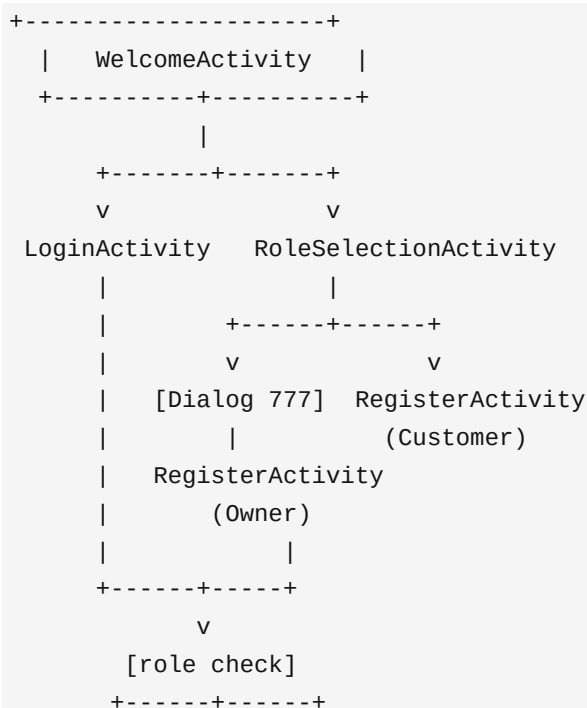
Description: Fields: season name, start date, end date, alert days in advance.

10. AddEditTaskActivity

Role: Task management

Description: Fields: title, description, assigned to, status (TODO/IN_PROGRESS/DONE), priority, due date.

2.3 Screen Flow Diagram



V					V				
MainActivity					CustomerMainActivity				
+---+---+---+---+					+---+---+---+---+				
DB	INV	SAL	TASK		SHOP	CRT	ORD	FAV	SUP

2.4 Project Class Descriptions (UML Overview)

View Layer (Activities + Fragments)

Contains all Activities and Fragments. Responsible only for displaying the UI and capturing user events. Never accesses data sources directly.

ViewModel Layer (9 ViewModels)

AuthViewModel, ProductViewModel, SaleViewModel, OrderViewModel, CartViewModel, FavoritesViewModel, TaskViewModel, SeasonViewModel, SupportViewModel. Each ViewModel exposes LiveData observed by the View layer.

Repository Layer (9 Repositories)

UserRepository, ProductRepository, SaleRepository, OrderRepository, CartRepository, FavoritesRepository, TaskRepository, SeasonRepository, SupportRepository. Each Repository communicates directly with Firestore.

Infrastructure Classes

AppDatabase, SessionManager, PermissionManager, CryptoUtil, FirebaseAuthHelper, NotificationHelper, LowStockReceiver, StorePilotApp.

3. Project Implementation

3.1 Separation of Logic and Display

The project strictly separates layers: the View layer (Activities & Fragments) handles only UI display and event capturing. The ViewModel + Repository layer handles all processing, validation, and computation. Data access occurs exclusively through Repositories.

3.2 Main Class Descriptions

1. AppDatabase

Role: Provides a shared thread pool for background operations.

- dbExecutor: ExecutorService with 4 threads.

Note: The database is Firestore only. AppDatabase is retained solely to provide the dbExecutor to other components.

2. SessionManager

Role: Singleton for managing the logged-in user in memory.

- Attributes: instance (static), loggedInUser.
- setLoggedInUser(User): marks a user as logged in.
- getLoggedInUser(): returns the current user.
- getUserRole(): returns the user's role string.
- isLoggedIn(): checks if a user is currently logged in.
- logout(): resets state and signs out from Firebase Authentication.

3. PermissionManager

Role: Static permission table defining which role can perform which action.

- permissionMap: Map<String, List<String>> linking permission to allowed roles.
- Constants: `MANAGE_USERS`, `MANAGE_PRODUCTS`, `VIEW_PRODUCTS`, `CREATE_SALE`, `VIEW_SALES_HISTORY`, `MANAGE_SEASONS`, `VIEW_ADMIN`.
- hasPermission(role, permission): checks if a specific role is authorized.
- currentUserHasPermission(permission): checks for the currently logged-in user.

```
// Restrict product management to Owner and Store Manager
permissionMap.put(MANAGE_PRODUCTS, Arrays.asList(OWNER, STORE_MANAGER));
```

4. CryptoUtil

Role: Password encryption using PBKDF2WithHmacSHA256.

- `generateSalt()`: creates a random 16-byte salt.
- `hashPassword(password, salt)`: hashes the password with the given salt.
- `verify(password, salt, hash)`: verifies a plaintext password against a stored hash.

5. FirebaseAuthHelper

Role: Wrapper for FirebaseAuth operations.

- `signUp(email, password, displayName, callback)`: creates a new Firebase Auth account and sets the display name.
- `signIn(email, password, callback)`: authenticates an existing user.
- `signOut()`: signs the current user out of Firebase.
- `getCurrentUid()`: returns the UID of the currently authenticated user.

6. StorePilotApp

Role: Application class - the initialization entry point for the app.

`onCreate()` performs three actions:

- `FirebaseApp.initializeApp(this)`: initializes Firebase.
- `NotificationHelper.createNotificationChannel(this)`: creates the notification channel.
- `scheduleLowStockAlarm(this)`: schedules the hourly stock check.

```
public static void scheduleLowStockAlarm(Context context) {
    AlarmManager alarmManager =
        (AlarmManager) context.getSystemService(Context.ALARM_SERVICE);
    Intent intent = new Intent(context, LowStockReceiver.class);
    PendingIntent pendingIntent = PendingIntent.getBroadcast(
        context, 0, intent,
        PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
    alarmManager.setInexactRepeating(
        AlarmManager.RTC_WAKEUP,
        System.currentTimeMillis() + AlarmManager.INTERVAL_HOUR,
        AlarmManager.INTERVAL_HOUR,
        pendingIntent);
}
```

7. NotificationHelper

Role: Creates the notification channel and sends low-stock alerts.

- `createNotificationChannel(context)`: creates the 'Low Stock Alerts' channel (required on Android 8+).
- `sendLowStockNotification(context, count)`: posts a HIGH-priority notification showing the number of low-stock products.

8. LowStockReceiver (BroadcastReceiver)

Role: BroadcastReceiver triggered by AlarmManager. Queries Firestore for products with quantity ≤ 5 .

Flow diagram of onReceive():

```

onReceive()
  |
  v
Query Firestore: products where quantity <= 5
  |
  +-- count > 0 --> NotificationHelper.sendLowStockNotification(count)
  |
  +-- count == 0 && testMode --> send demo notification + show Toast
  |
  +-- count == 0 && !testMode --> no action

```

- checkNow(context, testMode): static method for instant manual triggering. When testMode=true, sends a notification even if no products are low.

9. ProductRepository

Role: Manages all product operations against Firestore.

- getAllProducts(): returns LiveData backed by a real-time snapshot listener on the 'products' collection.
- insert(product): adds a new document to the 'products' collection.
- update(product): updates the document identified by firestoreId.
- delete(firestoreId): removes the document.
- getLowStockProducts(threshold): returns products with quantity below the threshold.

10. OrderRepository

Role: Manages customer orders including automatic sale creation on delivery.

Key method - updateOrderStatus:

```

public void updateOrderStatus(Order order, String newStatus) {
    db.collection("orders")
        .document(order.firestoreId)
        .update("status", newStatus);
    if ("DELIVERED".equals(newStatus)) {
        // Automatically create a sale record
        Sale sale = new Sale(null, 1, order.getTotalPrice(),
            System.currentTimeMillis(), null,
            "Order #" + order.getId() + " delivered");
        db.collection("sales").add(sale.toMap());
    }
}

```

- getActiveOrders(): returns only orders with status other than DELIVERED or CANCELLED. Filtering is done in Java to avoid Firestore composite index requirements.

11. SupportRepository

Role: Manages support chat conversations between customers and staff.

- Firestore structure: support/{customerId} (parent doc) + support/{customerId}/messages/{msgId}.
- sendMessage(): creates the parent document using SetOptions.merge() before adding the message to the subcollection. This ensures the conversation appears in the inbox.
- getMessagesForCustomer(): retrieves messages and sorts them by timestamp in Java (avoiding orderBy composite index).

3.3 Database

Database Overview

The project uses Firebase Firestore - a NoSQL cloud database. Data is organized into Collections and Subcollections.

Collection	Contents
users	User profiles (uid, username, email, role, localId)
products	Inventory items (name, category, quantity, price, costPrice)
orders	Customer orders (customerId, totalPrice, status, paymentMethod)
orders/{id}/items	Individual items within each order
sales	Sale transaction records
tasks	Employee tasks
seasons	Seasonal collections and promotions
carts/{userId}/items	Shopping cart items per customer
favorites/{userId}/items	Wishlist items per customer
support/{customerId}	Support conversation per customer
support/{customerId}/messages	Messages within the conversation

Operations on the Database

- Read: snapshot listeners wrap LiveData - the UI updates automatically whenever data changes in Firestore.
- Write: collection.add(map) for new records; document.update(field, value) for updates.
- Delete: document.delete() - only after user confirmation.

4. User Guide

4.1 System Requirements

- Android version: API 24 (Android 7.0 Nougat) and above.
- Internet connection: required for all operations (Firestore & Firebase Auth).
- Required permissions: INTERNET, POST_NOTIFICATIONS (Android 13+), RECEIVE_BOOT_COMPLETED.

4.2 Tested Versions

- Android 13 (API 33) - Pixel 7 emulator.
- Android 11 (API 30) - physical Samsung device.
- Android 8.0 (API 26) - emulator for compatibility testing.

4.3 First-Time Setup

1. Run the app from Android Studio on a device or emulator.
2. The welcome screen displays Sign In and Sign Up.
3. To create an owner account: tap Sign Up -> select Owner -> enter code 777 -> fill the form.
4. To create a customer account: tap Sign Up -> select Customer -> fill the form.

4.4 Usage Instructions

Adding a Product (Owner only)

1. Tap the Inventory tab.
2. Tap the + floating button.
3. Fill in: name, category, size, color, quantity, selling price, cost price.
4. Tap Save.

Placing an Order (Customer)

1. In the Shop tab, select a product -> tap Add to Cart.
2. Go to the Cart tab -> verify items.
3. Tap Checkout -> choose payment method and delivery address.
4. Tap Place Order -> receive a confirmation screen.

Managing an Order (Store Manager)

1. In the More menu, tap Orders.

2. Tap an order -> change its status (PENDING -> PROCESSING -> DELIVERED).

3. Changing to DELIVERED removes the order from the active list and automatically creates a sale record.

Testing Notifications (Test Mode)

1. Log in as Owner.

2. Open More menu -> tap [TEST] Low Stock Notification.

3. Pull down the device notification shade to see the alert.

Signing Out

- Customer: tap the lock icon in the top bar -> confirm in the dialog.
- Owner/Manager: More menu -> Logout.

4.5 User Messages

Situation	Message
Missing form fields	Please fill all required fields
Password mismatch	Passwords do not match
Password too short	Password must be at least 6 characters
Wrong login credentials	Incorrect email or password
Unauthorized screen access	Only the Owner can add or edit products
Order placed successfully	Order placed!
Low stock (notification)	X product(s) are running low. Tap to review.
Notifications blocked	Notifications blocked - enable in Settings

4.6 Constraints

- Password length: minimum 6 characters.
- Username: must be unique in the system.
- Email: must be in a valid format.
- Prices: positive values only.
- Stock quantity: cannot go below 0.

5. Personal Reflection / Summary

5.1 Work Process Description

I worked on StorePilot for approximately one year. The process began with the initial concept - combining a management system and a shopping app in one place - and evolved into a complex project containing over 90 Java files and dozens of XML layouts.

5.2 Successes

- Clean MVVM architecture: achieved complete separation between the View, ViewModel, and Repository layers, enabling easy maintenance and fast development.
- Successful Firebase integration: Firestore as a cloud database enables real-time UI updates without any polling mechanism.
- Permission system: PermissionManager is flexible and makes it easy to add new roles and permissions.
- Automatic notifications: AlarmManager + BroadcastReceiver work reliably even when the app is in the background.
- Automatic sale creation: when an order is marked as DELIVERED, the system automatically creates a sale record and updates revenue.

5.3 Challenges and Difficulties

- LiveData observer nesting: early on I registered observers inside other observers, creating multiple registrations per update. Solution: two separate observers, each storing data in a local field, with a merge function called when either updates.
- Firestore composite indexes: queries with orderBy on subcollections silently failed without a composite index in the Firestore Console. Solution: removed orderBy() from all subcollection queries and sort in Java instead.
- Support chat inbox empty: messages were saved in Firestore but did not appear in the inbox. Root cause: the parent support/{customerId} document was never created before messages were added to the subcollection. Fix: use SetOptions.merge() on the parent document before every sendMessage() call.
- Material3 style errors: the app theme is MaterialComponents (M2), not Material3 (M3). Had to ensure all layouts used Widget.MaterialComponents.* style prefixes.

5.4 Learning Process

- Learned design patterns in depth: Singleton, Observer, Repository, Adapter.
- Mastered Firebase end-to-end: Authentication, Firestore, differences between NoSQL and SQL.
- Improved understanding of security mechanisms: PBKDF2, salts, hashing.

- Understood the critical importance of background threads (ExecutorService) to keep the UI thread responsive.
- Learned to use CountdownLatch for synchronizing asynchronous Firestore calls on background threads.

5.5 Tools Taken Forward

- MVVM architecture: will be used in every future Android project.
- Firebase: a convenient and fast cloud service for small and medium projects.
- CountdownLatch for thread synchronization: an elegant solution for async operations that need to block and wait for a result.

5.6 Retrospective View

If I were to start the project again:

- I would use Hilt for Dependency Injection from the start instead of creating instances manually.
- I would implement Kotlin Coroutines instead of ExecutorService - cleaner and safer syntax.
- I would add Unit Tests systematically from day one.
- I would consider using Jetpack Compose instead of XML Views.

5.7 Future Improvements

- Firebase Cloud Messaging: real push notifications from the server, not just local.
- Firebase Storage: image upload for product photos.
- Multi-store support: manage multiple store branches under one account.
- Full i18n: Hebrew, English, and Arabic language support.

6. Bibliography

1. Google Inc. (2024). Android Developers Documentation. <https://developer.android.com/docs>
2. Google Inc. (2024). Firebase Documentation. <https://firebase.google.com/docs>
3. Google Inc. (2024). Firebase Firestore Guide. <https://firebase.google.com/docs/firestore>
4. Google Inc. (2024). Material Components for Android. <https://github.com/material-components/material-components-android>
5. Oracle Corporation. (2024). Java Cryptography Architecture (JCA) Reference Guide. <https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>
6. NIST. (2017). Special Publication 800-132 - Recommendation for Password-Based Key Derivation. National Institute of Standards and Technology.
7. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
8. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
9. JetBrains s.r.o. (2024). Android Studio User Guide. <https://developer.android.com/studio/intro>
10. Stack Overflow Community. (2024). Android development Q&A. <https://stackoverflow.com/questions/tagged/android>

7. Appendices

Appendix A - google-services.json

Located at app/google-services.json. Contains the Firebase project connection settings: API_KEY, APP_ID, PROJECT_ID. This file must NOT be committed to a public repository.

Appendix B - Project Folder Structure

```
StorePilot/
+-app/
  +-google-services.json
  +-build.gradle
  +-src/main/
    +-AndroidManifest.xml
    +-java/com/storepilot/
      | +-StorePilotApp.java
      | +-MainActivity.java
      | +-auth/      (WelcomeActivity, LoginActivity,
      | |              RegisterActivity, RoleSelectionActivity)
      | +-core/      (AppDatabase, SessionManager, PermissionManager,
      | |              CryptoUtil, FirebaseAuthHelper,
      | |              NotificationHelper, LowStockReceiver)
      | +-customer/  (CustomerMainActivity + 8 Fragments)
      | +-manager/   (OrderManagementFragment,
      | |              SupportConversationsFragment, SupportInboxFragment)
      | +-admin/     (UserManagementFragment)
      | +-dashboard/ (DashboardFragment)
      | +-inventory/ (ProductListFragment, AddEditProductActivity)
      | +-sales/     (SalesHistoryFragment, AddSaleActivity)
      | +-tasks/     (TaskListFragment, AddEditTaskActivity)
      | +-seasons/   (SeasonListFragment, AddEditSeasonActivity)
      | +-db/entities/ (9 Entity classes)
      | +-repositories/ (9 Repository classes)
      | +-viewmodels/ (9 ViewModel classes)
    +-res/
      +-layout/     (40+ XML files)
      +-drawable/   (icons and graphics)
      +-values/     (strings, colors, themes)
      +-menu/       (navigation menus)
```

Appendix C - Project Statistics

Metric	Count
Java files	90+
Activities	10

Fragments	18
ViewModels	9
Repositories	9
Firestore Collections	7
Lines of code	~6,000

Appendix D - Advanced Topics Implemented

Topic	Location in Project
Fragments (18 fragments)	Throughout the entire app
Threads (ExecutorService, 4 threads)	AppDatabase.dbExecutor
AlarmManager + BroadcastReceiver	StorePilotApp + LowStockReceiver
Notification + NotificationChannel	NotificationHelper
Firebase Authentication	FirebaseAuthHelper
Firebase Firestore (Remote DB)	All 9 Repositories
AlertDialog (owner code, logout)	RoleSelectionActivity, CustomerMainActivity
Async/Background work	dbExecutor in every Repository
LiveData + ViewModel (MVVM)	All ViewModel and Repository layers
PBKDF2WithHmacSHA256 hashing	CryptoUtil

Appendix E - Source Code

The full source code is available on GitHub:

<https://github.com/ahmadhijazys2/StorePilot>

All critical methods are documented with Javadoc comments. Single-line comments are used in less complex sections.

Example Javadoc - CryptoUtil.hashPassword()

```
/**
 * Hashes a plaintext password using PBKDF2WithHmacSHA256.
 *
 * @param password the plaintext password to hash
 * @param salt      the random salt to mix with the password
 * @return Base64-encoded hash string
 */
public static String hashPassword(String password, String salt) { ... }
```